Who Wrote It? Identifying LLMs from Their Responses

Rishi Raj | CMPSC 448 Machine Learning | October 2026

Abstract. I trained a TextCNN and a bidirectional LSTM to identify which of four LLM families produced a response. I used 8,000 examples selected from RAID, with the same prompts answered by ChatGPT, Llama, Mistral, and Cohere. On held-out prompts, the response-only CNN reached 75.1% macro-F1 and the LSTM reached 60.1%. Prompt text alone was uninformative in this matched design, and adding it to the response did not improve either model. I also tested two unseen task domains and measured simple style differences in the responses.

1. Problem, questions, and data

The task is to predict which LLM family wrote a generated response. Each labeled example has three essential fields: *LLM_Input* is the user's prompt, *LLM_output* is the model's answer, and *LLM_name* is the answer we want the classifier to predict. In this dataset, the same prompt was given to four different models. It therefore appears in four rows, with four responses and four different labels. The classifier is trained on labeled rows and evaluated on prompts it has never seen before.

The research questions

RQ1 asks: Can we identify which LLM generated a response? To answer it, I train a CNN and an LSTM using only *LLM_output* and measure how accurately they identify the model family on held-out examples.

RQ2 asks: Does the user's prompt help identify the LLM? For each architecture, I compare three separately trained versions: one sees only the prompt, one sees only the response, and one sees both. Using the same train, validation, and test rows for all three makes the comparison easier to interpret.

RQ3 asks: Do LLM fingerprints generalize across tasks or domains? A *fingerprint* here means a measurable pattern in responses, such as a recurring phrase or formatting habit. I train response-only models without news or poetry examples, then test them on the excluded domain. If performance falls, some of the learned patterns may depend on the writing task.

RQ4 asks: What characteristics distinguish different LLMs? This asks what those patterns look like in the first place. I compare response length, vocabulary diversity, formatting, and simple style-based classifiers to understand which observable cues might be helping classification.

Dataset source and curation

I used the labeled, non-adversarial RAID training CSV [1, 2]. RAID records the prompt, generated response, source model, task domain, and generation settings. Its Hugging Face dataset card lists an MIT license [1]. The four selected labels are ChatGPT, Llama, Mistral, and Cohere, each representing a distinct model family in this study. The eight selected domains are abstracts, books, news, poetry, recipes, Reddit, reviews, and wiki. These domains let the main experiment cover several kinds of writing while still giving RQ3 distinct tasks to hold out.

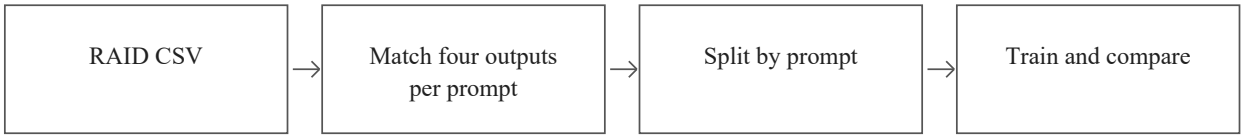


Figure 1. The data workflow. I keep the four responses to a prompt together from selection through splitting, so a prompt cannot appear in both training and test data.

The source file has 467,985 rows. I first kept the four model labels, the eight domains, examples without an adversarial attack, sampling decoding, and no repetition penalty. Fixing the generation settings matters because otherwise a classifier might learn a setting difference instead of a model difference. Those filters left 53,484 candidate rows, or 13,371 four-model source groups.

I then required every source group to contain one nonempty response from each model and the same nonempty prompt across all four models. I removed 121 groups whose prompt text repeated another group's prompt. This prevents the same question from appearing under different source IDs in different splits. The final selected sample has no repeated response texts. After these checks, 13,250 matched prompt groups were eligible.

To keep the experiment manageable and balanced, I selected 250 matched prompts from each domain using a deterministic hash order with seed 448. Each chosen prompt contributes four rows, one for each family. Thus every domain contributes 1,000 rows and the complete sample has 2,000 prompts and 8,000 rows. The prepared files retain the three assignment fields plus source model, source ID, record ID, and domain for auditing.

Split	Prompts	Rows	Rows per family
Training	1,400	5,600	1,400
Validation	304	1,216	304
Test	296	1,184	296

Table 1. I split matched source prompts within each domain at approximately 70/15/15. A complete four-row prompt group goes to one split, so source IDs and prompt text are disjoint across training, validation, and test. The same split is reused in all six primary experiments.

2. Preprocessing and models

I preserved letter case, punctuation, and line breaks because they might be part of a model's writing style. The tokenizer separates words, punctuation marks, and newlines. Its vocabulary comes only from the 5,600 training rows, using both prompt and response text. This produced 26,133 entries. Unknown words in validation or test data receive a shared unknown-token ID. I limited prompts to 96 tokens and responses to 512 tokens, based on training-set lengths; longer sequences are cut off and shorter ones are padded within a batch.

Each vocabulary token has an integer ID. An embedding layer uses that ID to look up a row of 128 numbers in a trainable table. The 128 numbers are features the model learns during training; they are not measurements supplied with the dataset or fixed vectors chosen in advance. The CNN and LSTM each learn their own embedding values as they learn the classification task.

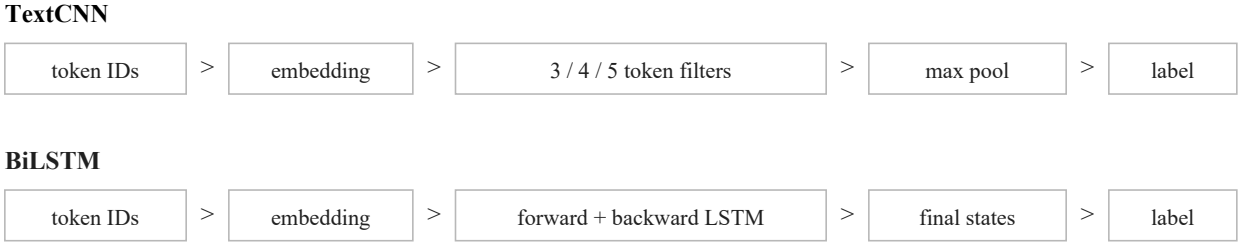


Figure 2. The two required classifiers. In both cases, the last layer predicts one of four model families.

TextCNN

The CNN slides small filters across the sequence of token vectors, much as an image CNN slides filters across pixels. Here the filters look for short text patterns instead of visual edges. Three sets of one-dimensional filters inspect spans of 3, 4, and 5 tokens, with 96 filters at each width. After ReLU, max pooling keeps the strongest match for each filter, wherever it occurred in the response. A dropout layer and linear classifier turn those pooled signals into four model scores. Padding is masked out before pooling.

Bidirectional LSTM

The LSTM reads the token vectors in order and carries a changing memory state from one position to the next. Its gates learn when to keep, update, or let go of information as more tokens arrive. This model reads in both directions, with 128 hidden units per direction. I combine the final forward and backward states, apply dropout, and predict the four labels. Packed sequences prevent padding from entering the recurrent calculation. Unlike the CNN's short filters, the LSTM can carry information across a longer part of the response.

Training and evaluation

Each architecture is trained three times: prompt only, response only, and prompt plus response. These are separate models because the available input changes. The combined version places a separator token between the prompt and response. All six runs use the same rows, label order, random seed (448), and training settings: AdamW, cross-entropy loss, batch size 32, learning rate 0.001, weight decay 0.01, dropout 0.3, and gradient clipping at 1.0. Training ran on a CUDA-enabled RTX 4060 GPU.

An *epoch* is one complete pass through the 5,600 training rows. With batches of 32, that is 175 batches per epoch. For each batch, the model predicts labels, compares them with the known labels using cross-entropy loss, and adjusts its trainable weights, including embeddings. I allowed at most 12 epochs. After each epoch, I measured macro-F1 on validation data without updating the weights and saved the model when that score improved. The *best epoch* is the pass with the highest validation macro-F1; it is not necessarily the final pass. Training stops after three epochs without improvement. I then loaded the saved weights and evaluated once on the test set.

3. Results for RQ1 and RQ2

Accuracy is the fraction of test examples classified correctly. Macro-F1 averages the four families' F1 scores equally, so a strong result for one family cannot completely hide weak results for another.

Model	Text provided	Accuracy	Precision	Recall	Macro-F1
TextCNN	Prompt only	0.250	0.250	0.250	0.203
TextCNN	Response only	0.754	0.759	0.754	0.751
TextCNN	Prompt + response	0.753	0.752	0.753	0.751
BiLSTM	Prompt only	0.250	0.062	0.250	0.100
BiLSTM	Response only	0.606	0.618	0.606	0.601
BiLSTM	Prompt + response	0.581	0.576	0.581	0.575

Table 2. Held-out test results on 1,184 examples. Precision and recall are macro averages across the four families. The full per-class scores and confusion counts are in the machine-readable results file.



Figure 3. Macro-F1 for the response-based settings. Dark bars use the response alone; light bars add the prompt.

RQ1: the response carries a usable signal

The output-only CNN scored 75.1% macro-F1, while the output-only LSTM scored 60.1%. Both did better than the 25% accuracy expected from always guessing one class in this balanced test set. The CNN was noticeably stronger in this experiment. Mistral was its hardest class: it correctly identified 164 of 296 Mistral examples and called 74 of them Cohere. That error pattern is why I kept confusion counts in the results file, although the assignment does not require a confusion-matrix figure.

RQ2: the prompt did not help here

Prompt-only accuracy was 25.0% for both architectures. This is a consequence of the data design: each prompt appears with all four model labels, so the prompt by itself cannot distinguish the label. Adding the prompt to the response left CNN macro-F1 effectively unchanged (both round to 75.1%) and lowered LSTM macro-F1 from 60.1% to 57.5%. The result does not mean prompts never matter. It says they did not improve these models on this matched sample.

I also tried an optional word-based comparison. TF-IDF represents a response by its words and two-word phrases, downweighting terms that occur in many training responses. Logistic regression learns a weight for each term and combines them to choose one of the four labels. Despite its name, it is a classifier here. This baseline reached 78.0% macro-F1 on the output-only test set; it does not replace the CNN or LSTM experiments.

4. RQ3: testing an unseen domain

For a direct cross-domain check, I selected two different task types before running the experiment: news and poetry. In each trial, I removed one domain from both training and validation, trained a new response-only CNN and LSTM on the other seven domains, and tested on the original test rows from the held-out domain. Each held-out test set contains 148 rows, 37 per family. I also evaluated the original eight-domain model on those exact same test rows. This comparison isolates the effect of excluding that domain during training, though it is only one seeded sample per domain.

Held-out domain	Model	Domain seen	Domain unseen	Change
News	CNN	0.663	0.468	-0.196
News	LSTM	0.523	0.322	-0.201
Poetry	CNN	0.704	0.326	-0.377
Poetry	LSTM	0.500	0.255	-0.246

Table 3. Macro-F1 on identical test rows. The 'domain seen' column uses the original model trained with that domain in its training split; the 'domain unseen' column uses a new model trained on the other seven domains.



Figure 4. Per-domain macro-F1. Light bars show the original model; dark bars show the model that never saw that domain during training or validation.

All four scores fell when the test domain was absent from training. Poetry had the largest drop for the CNN. That pattern suggests the classifiers relied partly on cues tied to the writing task, alongside whatever cues transfer across tasks.

Cohere had fairly high recall on the unseen domains (62% to 78%), but its precision was only 29% to 49%. The classifiers often predicted Cohere for texts from other families, so high recall alone does not show a stable Cohere fingerprint. Other class recalls changed sharply between news and poetry.

These tests are narrower than a full leave-one-domain-out study. A drop on an unseen domain would suggest that the classifier uses task-specific cues as well as more general model cues. A stable score would be evidence for transfer on that domain, but not proof of a universal fingerprint. The small domain-specific test sets and one random seed limit how precisely the changes can be estimated.

5. RQ4: what differs in the responses?

I looked at several properties of the training responses: token count, number of line breaks, vocabulary diversity, and punctuation per word. These are descriptive measures, not explanations of a model's internal behavior. I also trained small logistic regressions on output-only style features to see whether those properties alone were predictive.

Family	Median tokens	Median line breaks	Lexical diversity	Punctuation / word
ChatGPT	374.5	12.0	0.562	0.147
Llama	359.0	12.0	0.515	0.162
Mistral	253.0	8.0	0.557	0.147
Cohere	212.0	6.0	0.578	0.151

Table 4. Training-set medians. Lexical diversity is the fraction of distinct case-insensitive words in a response. Length affects this fraction, so values should be read alongside token counts.



Figure 5. Median response length in tokens, computed from training examples only.

The length difference also appears when I compare responses to the *same* training prompt: ChatGPT is longer than Cohere for 75.8% of 1,400 matched prompts, and Llama is longer than Mistral for 80.6%. This paired check reduces the chance that the length pattern is just a difference in assigned questions.

Output-only features	Test accuracy	Test macro-F1
Token count only	0.378	0.335
Eight style features	0.484	0.470
Style without length or line breaks	0.443	0.423
TF-IDF words and bigrams	0.783	0.780

Table 5. Exploratory models fit on training responses and evaluated on the original test split. The eight style features include token and word counts, average word length, lexical diversity, punctuation, line breaks, uppercase words, and digits.

Length alone reached 33.5% macro-F1. The eight style features reached 47.0%; removing both length and line breaks reduced that to 42.3%. This suggests structural features matter, but they account for less than the full word representation. The strongest positive TF-IDF phrases include 'sure here' for Llama, 'here is' for Cohere, and 'enjoy your' for Mistral. These coefficients show associations in this sample, not a causal fingerprint. Some high-weight terms are ordinary topic words.

6. What I learned

The biggest lesson for me was that a fair experiment starts with the data, not the model. Every prompt in this sample has an answer from all four LLM families. Once I understood that structure, the 25% prompt-only accuracy made sense: the prompt cannot tell the classifier which of its four answers it is looking at. I also understood why splitting individual rows would be misleading. Keeping each four-response prompt group together, checking duplicates, and holding the generation settings constant made the test results much easier to trust.

Implementing the networks made ideas from machine learning more concrete. I could follow the path from text to tokens, token IDs, learned 128-number embeddings, and finally four class scores. Before this project, I mainly associated CNNs with images and expected an LSTM to be stronger on text because it reads in sequence. The response-only CNN reached 75.1% macro-F1 here, compared with 60.1% for the LSTM. Seeing that result taught me to test an architectural expectation instead of treating it as a rule. The CNN's short filters are well suited to recurring phrases and formatting, although this experiment alone does not prove why it won.

I also learned what the validation set is doing during training. An epoch is one pass through the training rows, and the best epoch is chosen using validation macro-F1. The test set is then used for the final score. Running separate prompt-only, response-only, and combined models on the same splits helped me see how one controlled change in the input affects the result. It made RQ2 a comparison I could explain, rather than just three unrelated numbers.

The simple baseline was another useful surprise. TF-IDF with logistic regression reached 78.0% macro-F1 on responses, above both neural models in this sample. That result showed me why a baseline belongs in an ML project: it tells us whether a more complicated model actually adds value. The style-only features reached 47.0%, so length, punctuation, and formatting carry some information, while words and short phrases carry more. Comparing response lengths for the same prompt also gave me a better way to examine style without confusing it with different assigned questions.

Finally, the news and poetry holdouts showed that a good test score on familiar tasks does not guarantee the same result on a new kind of writing. For example, the CNN's poetry macro-F1 fell from 70.4% when poetry appeared in training to 32.6% when it was excluded. I came away with a clearer way to present results: state what was measured, check what changed between experiments, and connect the conclusion to that evidence. If I continued the project, I would repeat the runs with more seeds and hold out more domains to see how consistently these patterns appear.

References

[1] RAID dataset card, Hugging Face. huggingface.co/datasets/liamdugan/raid

[2] Dugan et al. 2024. RAID: A Shared Benchmark for Robust Evaluation of Machine-Generated Text Detectors. *Proceedings of ACL*. aclanthology.org/2024.acl-long.674

[3] CMPSC 448 midterm assignment. *Who Wrote It? Identifying LLMs from Their Responses*. Course PDF, modified September 17, 2026.